

ANO
2024



UNINTER

CADERNO DE RESPOSTAS DA
ATIVIDADE PRÁTICA DE:

NoSQL

ALUNO: (ESTHER MARA BALDUÍNO DE PAULO RU:)

Caderno de Resposta Elaborado por:
Prof. MSc. Guilherme Ditzel Patriota

Atividade Prática – NEO4J

Questão 01 – IMPORTAÇÃO DOS ARQUIVOS JSON E CRIAÇÃO DE NÓS E ARESTAS COM BASE NOS DADOS DOS ARQUIVOS (Usar quantas páginas forem necessárias)

O QUE FAZER: Após configurar seu banco de dados em branco (novo DBMS em versão 4.*) e colocar no mínimo 3 e máximo 10 arquivos sequenciais do trabalho (dentre os 500 arquivos JSON disponibilizados) na pasta Import, crie um comando na linguagem do banco de dados Neo4j (Cypher), usando a biblioteca de importação de dados APOC, que leia os arquivos JSON desta pasta e crie os nós e relacionamentos com base nos dados neles. Faça a separação dos nós de mensagem em Tweet (mensagens originais), Retweet (mensagens repostadas), Quoted (mensagens que citam outras mensagens), Replied_to (mensagens de resposta à outras mensagens) com uso do campo `data[x].ref_tweet.type` (CUIDADO! Este campo só aparece no JSON de mensagens não originais e o uso de UNWIND para acessá-la pode impedir a criação dos nós de mensagens originais). Apenas com esta separação será possível resolver a questão 02 do trabalho.

Observação Importante: Seu banco de dados precisará conter todas as informações para resolver as questões 02 e 03. Leia elas antes, para entender o que você deseja fazer em cada uma e quais os nós, relacionamentos e atributos você irá importar dos arquivos JSON para conseguir resolver todo o trabalho sem a necessidade de recriar todo o seu banco de dados apenas para uma questão.

- I. Apresentação dos comandos (apenas query Cypher) usados (não esquecer do identificador pessoal/seu RU como parte do seu código, como um nome de atributo dos nós ou dado de um atributo de nós). **Nenhum dos comandos apresentados aqui pode conter a palavra RETURN (Peso na nota: 12.5%):**



```
1 // Meu RU
2 MERGE (ru:RU {RU: "RU-4328984"});
3 // Carregar todos os arquivos JSON da pasta import
4 CALL apoc.load.directory('*.json') YIELD value
5 UNWIND value AS arquivo
6
7 // Ler o conteúdo de cada arquivo JSON
8 WITH arquivo
9 CALL apoc.load.json(arquivo) YIELD value
10 UNWIND value.data AS tweet
11
12 // Criar nós de acordo com o tipo da mensagem
13 MERGE (t:Tweet {tweet_id: tweet.id})
14 ON CREATE SET
15     t.text = tweet.text,
16     t.created_at = datetime(tweet.created_at),
17     t.conversation_id = tweet.conversation_id,
18     t.lang = tweet.lang,
19     t.source = tweet.source
20
21 // Identificar o tipo de mensagem (Tweet, Retweet, Quoted, Replied_to)
22 FOREACH (ref_tweet IN coalesce(tweet.referenced_tweets, [])) |
23     CASE ref_tweet.type
24     WHEN 'retweeted' THEN
25         MERGE (r:Retweet {tweet_id: tweet.id})
26         ON CREATE SET r.text = tweet.text, r.created_at = datetime(tweet.created_at)
27     WHEN 'quoted' THEN
28         MERGE (q:Quoted {tweet_id: tweet.id})
29         ON CREATE SET q.text = tweet.text, q.created_at = datetime(tweet.created_at)
30     WHEN 'replied_to' THEN
31         MERGE (rp:Replied_to {tweet_id: tweet.id})
32         ON CREATE SET rp.text = tweet.text, rp.created_at = datetime(tweet.created_at)
33     ELSE
34         MERGE (t:Tweet {tweet_id: tweet.id})
35     END
36 )
37
38 // Criar o nó do autor e relacionar com a mensagem
39 MERGE (u:User {user_id: tweet.author_id})
40 MERGE (u)-[:TUITOU]->(t)
```

RU: 4328984

```
42 // Adicionar hashtags
43 WITH t, tweet, tweet.entities.hashtags AS hashtags
44 UNWIND hashtags AS hashtag
45 WITH t, apoc.text.replace(apoc.text.clean(hashtag.tag), '[^a-zA-Z0-9]', '') AS cleanedHashtag
46 MERGE (h:Hashtag {tag: cleanedHashtag})
47 MERGE (t)-[:POSSUI]->(h)
48
49 // Adicionar URLs
50 WITH t, tweet, tweet.entities.urls AS urls
51 UNWIND urls AS url
52 MERGE (u:URL {url: url.expanded_url})
53 MERGE (t)-[:INCLUI_URL]->(u)
54
55 // Relacionar mensagens retweetadas, citadas ou respondidas
56 FOREACH (ref_tweet IN coalesce(tweet.referenced_tweets, [])) |
57     MERGE (ref:Tweet {tweet_id: ref_tweet.id})
58     MERGE (t)-[:REFERENCIA]->(ref)
59 );
```

Figura 1: Nesta questão, realizei a importação dos dados dos arquivos JSON e a criação dos nós correspondentes no banco de dados Neo4j. Através do uso da biblioteca APOC, importei os tweets, hashtags, usuários e relacionamentos entre eles.

Tweet: Representa cada mensagem original, com atributos como texto, data de criação, ID da conversa e idioma.

User: Representa o autor do tweet, relacionado ao nó de Tweet através da relação [:TUITOU].

Hashtag: Representa as hashtags mencionadas nos tweets, relacionadas aos nós de Tweet através da relação [:POSSUI].

URL: Representa as URLs incluídas nos tweets, relacionadas aos nós de Tweet através da relação [:INCLUI_URL].

- II. Apresentação dos prints do resultado (não esquecer do identificador, seu RU). Estes prints devem ser de cada tela do Neo4j Browser após a execução bem-sucedida de cada comando (não mostrar nenhum grafo nesta parte, apenas as telas de execução dos comandos da parte I. O uso de RETURN zerará a nota desta parte) (Peso na nota: 12.5%):**

Figura 2: Exibe o resultado da importação dos dados JSON no banco de dados Neo4j. Após a execução do comando Cypher, o banco de dados foi populado com nós e relacionamentos correspondentes a tweets, usuários, hashtags e URLs.

Atividade Prática – NEO4J

Questão 02 – DESCOBERTA DA HASHTAG PRINCIPAL (Usar quantas páginas forem necessárias)

ENUNCIADO: Você deve criar e executar um comando Cypher em seu banco de dados para descobrir qual hashtag está presente em todas as mensagens originais (excluindo mensagens de retweet, citação e resposta). Este comando **não deve** fazer uso da biblioteca **APOC**. Caso seu comando tente retornar mais de 300 nós, a configuração padrão do Neo4j Browser impedirá a exibição de mais de 300 nós.

Seu comando **não pode** ser **MATCH (n) RETURN n;**.

- I. Apresentação do comando (apenas query Cypher) usado (não esquecer do identificador pessoal no comando, seu RU). **Este comando deve conter ao menos uma dupla MATCH/RETURN (sugerimos uso de subquery para resolução completa) (Peso na nota: 12.5%):**

```
// Encontrar a hashtag mais usada em tweets originais (sem retweets, respostas ou citações)
MATCH (t:Tweet)-[r:POSSUI]->(h:Hashtag)
WHERE NOT t.tipo_ref IN ['retweeted', 'quoted', 'replied_to'] // Filtrar apenas tweets originais

// Contar quantos tweets estão relacionados a cada hashtag
WITH h, COUNT(r) AS contagem
ORDER BY contagem DESC // Ordenar pelas hashtags mais usadas
LIMIT 1 // Selecionar a hashtag mais comum

// Buscar os tweets conectados a essa hashtag e o RU
WITH h, contagem
MATCH (h)-[r:POSSUI]-(t:Tweet), (ru:RU {RU: "RU-4328984"}) // Relacionar com tweets e RU

// Retornar o grafo com a hashtag principal, tweets e RU
RETURN h AS HashtagPrincipal, t AS Tweet, r, contagem, ru
LIMIT 15; // Limitar a quantidade de nós para evitar poluição visual
```

Figura 3: Aqui, o objetivo era encontrar a hashtag mais usada em tweets originais, ou seja, aqueles que não são retweets, respostas ou citações. Utilizamos uma consulta no Neo4j para filtrar os tweets que não tinham referências a outros tweets e contarmos a frequência de uso das hashtags nesses tweets.

Filtragem: Excluímos retweets, respostas e citações para focar apenas em tweets originais.

Contagem: Contamos quantas vezes cada hashtag apareceu em tweets originais.

Hashtag Principal: Após ordenar pela quantidade de ocorrências, identificamos a hashtag mais frequente, que provavelmente foi usada como filtro para a coleta dos dados.

- II. Apresentação do grafo gerado contendo apenas 1 nó de hashtag ao centro (a sua resposta) e ao menos mais 10 nós de mensagens relacionadas a este nó de



Atividade Prática – NEO4J

Questão 02 – DESCOBERTA DA HASHTAG PRINCIPAL (Usar quantas páginas forem necessárias)

hashtag. O print deve conter o grafo sem zoom e a legenda de tipos de nós e cores geradas pelo neo4j, incluindo a informação da quantidade de nós de Hashtag mostrados (não esquecer do identificador pessoal, seu RU) **(Peso na nota: 12.5%)**:

Figura 4: Exibe um grafo de visualização criado no Neo4j para mostrar a hashtag mais usada em tweets originais, sem retweets ou respostas. O grafo visualiza a hashtag principal no centro, conectada a uma série de tweets originais que a utilizaram.

- III. Responda à pergunta: Qual foi a hashtag usada como filtro para coleta dos dados analisados? (Esta hashtag só estará presente em nós do tipo Tweet e não em Retweet. Sua resposta deve conter apenas 1 palavra com o texto da hashtag) **(Peso na nota: 12.5%)**?

Resposta (apenas 1 palavra): #issoaglobonaomostra



Atividade Prática – NEO4J

Questão 03 – ANÁLISE DOS DADOS SEGUNDO VIÉS A SUA ESCOLHA (Usar quantas páginas forem necessárias)

ENUNCIADO: Usando o mesmo banco de dados já criado na questão 01 e usado na questão 02, busque alguma informação que você julgue relevante nos dados (seu comando **não pode** ser **MATCH (n) RETURN n**; nem pode conter a biblioteca **APOC**). Sua tarefa aqui é analisar os dados do banco e definir qual informação você gostaria de obter e que tenha potencial de gerar um grafo com 10 ou mais nós interligados entre si. Sua análise deve responder à uma pergunta clara, como por exemplo:

- Qual o dispositivo mais usado para tuitar? (depende de seu banco de dados já possuir os nós de equipamentos usados, criados na questão 01).
- Qual a Hashtag que menos foi usada?
- Qual o usuário mais movimentou a rede? (depende de você ter criado nós de usuário na questão 01)
- Quais os usuários mais citados? (depende dos seus nós de mensagem possuírem esta informação ou de relacionamentos terem sido criados para este fim).

- I. Apresentação dos comandos (apenas queries Cypher) usados por você para realizar sua análise (não esquecer do identificador pessoal, seu RU) **(Peso na nota: 12.5%)**:

```
1 // Encontrar o usuário que mais movimentou a rede (mais postou tweets)
2 MATCH (u:User)-[:TUITOU]->(t:Tweet)
3
4 // Contar quantos tweets cada usuário postou
5 WITH u, COUNT(t) AS contagem
6 ORDER BY contagem DESC // Ordenar pela quantidade de tweets postados
7 LIMIT 1 // Selecionar o usuário mais ativo
8
9 // Buscar os tweets do usuário e o RU
10 WITH u, contagem
11 MATCH (u)-[:TUITOU]->(t:Tweet), (ru:RU {RU: "RU-4328984"}) // Relacionar com os tweets e o RU
12
13 // Retornar o grafo com o usuário, seus tweets e o RU
14 RETURN u AS UsuarioAtivo, t AS Tweet, contagem, ru
15 LIMIT 20; // Limitar a quantidade de nós para evitar poluição visual
```

RU:4328984

Figura 3: Nesta questão, escolhi analisar qual usuário postou mais tweets. Através de uma consulta no Neo4j, buscamos contar quantos tweets cada usuário publicou e identificar o mais ativo. O relacionamento entre os nós User e Tweet, criado na importação, foi usado para realizar essa contagem.

- II. **Apresentação do print do resultado, podendo ser uma tabela ou um grafo. O print deve conter o resultado e o comando executado. (não esquecer do identificador, seu RU) (Peso na nota: 12.5%):**

Figura 4: Grafo que mostra o usuário mais ativo e os tweets que ele postou.

- III. **Explique qual foi a análise realizada, incluindo sua linha de raciocínio para criação da query Cypher e qual era sua expectativa de resultado antes da análise, comparando-a com o resultado realmente obtido após execução do comando (Peso na nota: 12.5%).**

Resposta:

O objetivo foi realizar uma análise específica dos dados que já estavam no banco de dados após a importação dos arquivos JSON. A análise realizada foi focada em identificar qual usuário postou mais tweets na rede social. Para isso, usei o relacionamento entre usuários e tweets, e criei uma query Cypher para contar os tweets e encontrar o usuário mais ativo.

A ideia principal era simples: eu sabia que cada tweet estava relacionado ao seu autor através do relacionamento [:TUITOU], criado no código durante a importação dos dados JSON. Esse relacionamento conecta os nós User (usuários) aos nós Tweet (tweets). Então, pensei em contar quantos tweets cada usuário postou e ver quem postou mais.

Identificar a relação entre usuários e seus tweets: O código importava as mensagens e criava uma relação [:TUITOU] entre os nós User (autor) e Tweet (mensagem). Com isso, foi possível contar o número de tweets que cada usuário postou.

Contar a quantidade de tweets por usuário: Utilizei a função COUNT () para contar quantos tweets estavam conectados a cada usuário.

Ordenar os usuários pelo número de tweets: Depois de contar os tweets, eu precisava ordenar esses números de forma decrescente para que o usuário com mais tweets aparecesse primeiro.

Selecionar o usuário mais ativo: Finalmente, usei LIMIT 1 para pegar apenas o usuário com o maior número de postagens.



Antes de rodar o comando, minha expectativa era que um pequeno grupo de usuários seria responsável pela maioria das postagens, o que é comum em redes sociais. Normalmente, há alguns "super usuários" que postam muito mais do que outros, então eu esperava encontrar um usuário com um número significativamente maior de tweets em comparação com a maioria.

Eu também esperava que o resultado mostrasse uma disparidade significativa, com um ou dois usuários dominando o número de tweets.